Switching-Aware Precision Selection on Power-Capped GPUs: Mantissa Truncation versus INT8 Emulation

Koying Chen
Department of Computer Science, National Tsing Hua University, Hsinchu, Taiwan

***Abstract*—When a GPU runs at its power limit, the energy of each operation sets the clock. Zeroing low-order mantissa bits is known to lower GPU power in microbenchmarks; we ask whether doing it in software, on real data, is a usable optimization. On an NVIDIA GH200 cluster and two 8-GPU AMD MI350X nodes, the saving comes from bit flips between neighbouring operands: on MI350X, with values fixed, reordering alone changes the energy per FP64 operation by 8–9%, and flips counted by direction predict held-out operand layouts ($R^2 = 0.93$). A first-order power model, calibrated by one locked-clock GEMM on an application's own operands, predicts its speedups under caps within 1.2 percentage points on average. MI350X is power-bound at its stock limit, so in-program truncation speeds up FP64 HPL, HPL-MxP, a Gaussian-process likelihood and DF-MP2 by about 5–10% at default settings, each passing its accuracy check. Against INT8 emulation of FP64 the faster choice depends on the inner dimension, the cap and the tolerance; a per-call selector fitted on GEMM measurements alone picks it in 98% of held-out GEMM cases on MI350X, 83–98% on GH200 and 22 of 24 Gaussian-process runs (always one method: at most 80% and 12 of 24). On data with wide per-row spans, emulation can lose small entries while its normwise error stays small; a componentwise FP64 spot check detects the loss, and one reference run per application decides whether it matters. Only truncation applies to BF16 and FP8.**

***Index Terms*—power capping, energy efficiency, floating-point precision, mantissa truncation, GPU, mixed precision, HPL, emulation**

I. Introduction

Power is becoming the ceiling on delivered performance. Data centers cap rack power, competitions cap cluster power, and accelerators ship with package limits that their own workloads reach: the AMD Instinct MI350X holds FP64, BF16 and FP8 matrix multiplication at its default 1000 W limit 82–100% of the time in our measurements. Under a limit the power manager lowers the clock until the device fits, so the speed of a program depends on how much energy each of its operations consumes.

The effect we build on is known. DGEMM on random operands draws 67% more power than on constant ones on A100 [1]. Gregersen et al. showed that zeroing the least significant bits of GEMM inputs lowers power, attributed it to fewer bit flips and proposed that compilers or schedulers exploit it [2]; a zero-input DGEMM on GH200 draws 563 instead of 898 W [3]; and an informal measurement showed that predictable GEMM data also runs faster under a lower power limit [4]. Hardware designs narrow multipliers or mask mantissas at unit inputs [5]–[8]. What these works leave open is whether the effect can be turned into a usable optimization: whether software truncation of *real* operands speeds up applications on production GPUs of different vendors while each application's accuracy check passes; whether the gain under a cap can be predicted rather than swept; and when it is the better choice than a format change such as INT8 emulation of FP64 [9], [10], whose speed also depends on the power limit. We answer these questions on two vendors' GPUs, ending in a selection we call *switching-aware* because it accounts for the data-dependent switching energy of each method's operands, which sets the clock under a cap:

1) **Where the gain comes from and how much of it can be predicted.** Fills that stop the discarded bits toggling without making them zero retain 65–97% (ones) and 41–92% (fixed pattern) of the saving in matrix engines, FP64 vector units and memory copies of both vendors. With values and set bits held fixed, reordering operands alone changes the energy per operation by 8–9 points, and bit flips between neighbouring operands, counted separately across rows and along k, predict the energy of held-out layouts (leave-one-out $R^2 = 0.93$, against 0.79 for all flips and 0.34 for set bits; Section IV). If truncation only changes the switched capacitance, the truncated program under cap P behaves like the native program under $P_0 + (P - P_0)/R$, with P_0 the measured idle power. Calibrated by one locked-clock run per truncated variant (on GH200, which cannot lock clocks, one capped run), this first-order model predicts 42 cap–format GEMM points per platform with mean errors of 1.5 and 2.1 percentage points (at most 4.6 and 10.4). Calibrated from operand bit flips alone it predicts FP64 GEMM speedups within 1.2 points, a test of only three predicted R values, and overpredicts DF-MP2 by 4.4. Calibrated by a locked-clock GEMM on the applications' own operands, weighted by its share of the dynamic energy, it predicts two applications not used to build it within 1.2 points on average, as closely as a locked-clock run of the whole truncated program (0.8–1.7 over repeated runs) (Section V).
2) **Gains that hold in real codes with their own accuracy checks.** On a GPU that is power-bound at its default limit (MI350X), in-program truncation speeds up HPL, HPL-

MxP (on its diagonally dominant test matrix and on a harder one), DF-MP2 and a Gaussian-process likelihood by 4.6–10%, replicated on a second node; on GH200 the same codes gain under caps and save energy at the default. Safe bit counts must be validated per platform (Section VI).

3) **When to choose INT8 emulation instead, per call.** At equal accuracy emulation is faster for long inner dimensions at generous caps, truncation for short inner dimensions, tight caps and strict tolerances; limited memory and wide per-row exponent spans also favor truncation, and of the two only truncation applies to BF16 and FP8. A per-call selector fitted on GEMM measurements alone picks the faster method in 97.7% of held-out MI350X GEMM cases (always emulating or always truncating: 52 and 48%) and, refitted on GH200, in 98.3 and 83.3% at tolerances of 10^{-9} and 10^{-11} (always truncating: 60 and 80%). It matches DF-MP2, where emulation wins 99.3% of the calls, and is right in 22 of 24 Gaussian-process runs whose block size moves the crossover with the cap, against 12 for either fixed choice (Section VII).

II. Background and Related Work

Input-dependent power. Gregersen et al. varied value distribution, bit similarity and sparsity of GEMM inputs on A100, H100 and V100 and saw power differ by almost 40%, but avoided throttling [2]; H. He swept the A100 power limit [4] and Antepara et al. measured GH200 [3], [11], both with all-zero versus random inputs; Lucas and Juurlink modeled ALU power by the number of toggles [12]. None truncates real data, evaluates accuracy or applications, or compares vendors. For density fitting, Huang et al. adapt the precision of INT8 emulation to the SCF state [13]; that changes the format, not the bits fed to FP64 units.

Power-constrained HPC. Manufacturing variation makes processors differ under the same power bound [14], [15]; overprovisioning [16] and power-limit tuning [17] decide how power is distributed. We lower the energy each operation needs, so the two compose.

Masking, precision tuning and data movement. Mantissa masking in hardware saves energy in multipliers [5], shaders [6], [18], [19], baseband processors [7] and register files [20]; DRAM and HBM energy depend on toggling [21]–[23]. Precision tuning [24], mixed-precision storage [25], [26] and transprecision computing [27] change types or bytes; zeroing changes neither. Low-precision LU with refinement underlies HPL-MxP [28], [29], and bit-information analysis can estimate needed bits from data [30].

FP64 emulation and format changes. The Ozaki scheme splits a product into exactly computed low-precision slices [31], [32]; Ozaki scheme II uses modular arithmetic [9] and is implemented in GEMMul8 for CUDA and HIP [10]; cuBLAS ships a fixed-point emulation with an automatic mode that picks the bit count from the exponent span [33], [34]. Tile mixed precision converts whole tiles of geostatistics factorizations to lower formats [35]–[39], and convergence-driven schemes switch formats or slice counts by solver state [13], [40]. All of these change the format; truncation keeps the original units, so the two can compose.

III. Method and Experimental Setup

Truncation. We keep the top b of the t fraction bits of an operand (52 for FP64, 7 for BF16, 3 for FP8 E4M3) and zero the rest, by truncation (a bit mask) or round-to-nearest (add half a unit of the last kept bit to the bit pattern first, so ties round away from zero and a carry out of the fraction raises the exponent, which is the correctly rounded result); the sign is untouched. All bit counts in this paper are kept fraction bits; for HPL, whose parameter counts significand bits, 35 fraction bits correspond to its setting 36. HPL rounds to nearest even; applications round to nearest; GEMM microbenchmarks and HPL-MxP truncate. For uniform operands the relative error of $C = AB$ (normwise, $\|\hat{C} - C\|_F / \|C\|_F$, the error measure used throughout) is $\approx \alpha 2^{-b}$ with $\alpha \approx 0.27$ (nearest) or 0.70 (truncation), independent of the inner dimension; truncation's error is a systematic shrinkage, which solvers with residual correction tolerate but accumulating processes do not. On real data α ranges from 0.07 to 9.6 with cancellation, so tolerable bit counts must be measured per application. With $p_n(P)$ and $p_r(P)$ the native and truncated throughput under cap P, the equivalent saving $\Delta W(P) = p_n^{-1}(p_r(P)) - P$ is the extra cap the native program needs to match the truncated one, interpolated along the measured native curve. For plane-wave DFT we tie the bit count to the Davidson threshold *ethr*: $b = \mathrm{clamp}(\lceil -\log_2 ethr \rceil + m, 16, 52)$ with margin $m = 0$.

GH200. Fourteen nodes with one GH200 480GB superchip each (Hopper GPU, maximum SM clock 1980 MHz, FP64 tensor peak 67 TFLOP/s [41]), 200 Gb/s InfiniBand, NVHPC 26.5 (CUDA 13.2). The module cap (GPU, CPU and memory) is set from 200 to 1000 W; the default is also the maximum.

MI350X. Two nodes (n01, n02) with eight MI350X OAM GPUs each (gfx950, 288 GiB HBM3E, maximum clock 2200 MHz), ROCm 7.2.3, Open MPI 5.0.7 with UCX. The per-GPU package limit is set from 1 to 1000 W; the default is the maximum; idle power is 244–275 W per GPU.

Workloads. GEMM microbenchmarks with cuBLAS or rocBLAS/hipBLASLt on operands uniform in $[-1, 1)$ ($m = n = 32768$; inner dimension $k = 2048$ for FP64, 6144 or 8192 for BF16 and FP8). HPL and HPL-MxP: on GH200 8 nodes (HPL tuned, $N = 286720$; HPL-MxP $N = 491520$); on MI350X one node of 8 GPUs (HPL $N = 540672$, 1×8; HPL-MxP $N = 1034752$, 7.6 PFLOP/s natively); HPL-MxP uses a diagonally dominant test matrix, which its rules allow for testing only [42], and one run uses a matrix that is not (Section VI-B). Quantum ESPRESSO 7.6 [43] (Si 1000 at 550 W, Au 112 at 450 W; GH200). DF-MP2 for taxol def2-TZVP (113 atoms): GPU4PySCF [44] on GH200; on MI350X, where GPU4PySCF does not run, the same three-index tensor built by PySCF on the CPU and contracted on the GPU (energy within 2×10^{-13} E_h of PySCF and 7.5×10^{-8} E_h of

TABLE I

WHERE EACH CODE TRUNCATES, HOW OFTEN, THE KEPT FRACTION BITS (GH200 / MI350X; ALL CODES ROUND TO NEAREST EXCEPT HPL-MXP), AND THE EXTRA WORK. ALL TIMINGS INCLUDE IT.

Code	Operand, frequency	Bits	Extra work
HPL	L, U panels, each update	35 / 35	pass over panels
HPL-MxP	BF16 storage, each write	3 / 3	fused
QE	Davidson blocks, each step	16–42 / —	in place
DF-MP2	tensor B, once	20, 32 / same	one pass, $< 0.1\%$
GP	panel L_{21}, each update	28, 36 / same	fused

GPU4PySCF). An ExaGeoStat-like Gaussian-process likelihood on both ($n = 80000$, Matérn-3/2, blocked Cholesky with one FP64 DGEMM per panel). Truncation happens inside each program; GEMMul8 [10] and cuBLAS emulation replace FP64 GEMMs through a hook or the math mode; cuBLAS emulation's bit count is its own mantissa setting. GEMMul8 3.2.0 runs through its hook (HPL, DF-MP2 and GP cap sweeps, Fig. 3), 3.5.4 through its direct API (shape sweep, selector, DF-MP2 per call, GP block sizes); both run DF-MP2 in 12.8–13.0 s. Table I lists where each code truncates.

Measurement. GH200: nvidia-smi samples every 250–500 ms, and energy integrates 1-s averaged module power. MI350X: a sampler reads all eight GPUs every 100 ms, including a 15.26-μJ hardware energy counter (energy is its difference across a window) and the residency of power and thermal limiting. Every run records the cap in effect. Variants run in ABBA order on the same nodes or GPUs after a cool-down; on MI350X all 8 GPUs run the same configuration at once.

A pitfall: fans and temperature. On MI350X the chassis fans react to the previous run: the next run can be up to 15 °C cooler and 4.5% faster, because cooler silicon leaks less, and interleaving alone does not remove this (it made 4 kept bits of HPL-MxP look faster than 3 at 800 W in one job). Regressing performance on the temperature during a run would also remove the cooling that truncation itself causes; regressing on the temperature at its start does not see the fan state (HPL runs start after a cool-down to a common temperature). We report raw paired ratios (what a user obtains), start-adjusted values beside them, and controlled checks with a common thermal load before every run: GEMM at steady state stays within 1.5 points of the short windows; HPL-MxP at 800 W restores the raw order (3 bits $+10.4 \pm 2.8$, 4 bits $+6.0 \pm 3.5\%$); DF-MP2 and the likelihood at 1000 W, with ABBA and BAAB rounds alternated, give 12.6 ± 0.1 and $7.3 \pm 0.1\%$ (20 and 28 bits) against 10.3 and 7.4 raw, 11.5–11.8 and 8.0–8.2 at equal start and 13.3–14.2 and 7.1–7.3 at equal mean temperature. The spread between thermal methods (DF-MP2 10.3–14.2, the likelihood 7.1–8.2%) exceeds every statistical interval we report, so we give raw values as the main result, which is what a user obtains and, for DF-MP2, the most conservative. Fans can be read but not set with our access.

Statistics. Each speedup is a paired ratio on the same node, cap and ABBA round. Measurements that share a chassis or a job are not independent, so on MI350X the unit is one measurement window of all 8 GPUs (GEMM), one node run (HPL, HPL-MxP) or one round of all 8 GPUs (GP, DF-MP2), and every MI350X experiment is repeated on the second node. On GH200 the units are node–pair samples from two node sets (HPL, HPL-MxP) or three nodes (GEMM), which treats pairs within a job as independent; those intervals may be too narrow. Even on MI350X, units within one job may remain correlated, so the intervals describe how stable a gain is on these nodes, not across a fleet; a mixed-effects model (random intercept per block or round) gives the same estimates within 0.1 point and intervals of similar width.

IV. WHY FEWER MANTISSA BITS SAVE POWER

Dynamic power is roughly $\alpha_{\mathrm{sw}} C V^2 f$ with a data-dependent switching activity α_{sw}. Under a cap the power manager lowers f (and V) until the device fits, so a lower α_{sw} becomes clock; below the cap it appears as lower power. Both occur on both platforms. On GH200 at its default 1000 W module cap the native FP64 GEMM is capped 95% of the time at 1822 MHz; with 0 kept bits it runs at the maximum 1980 MHz while module power falls from 984 to 707 W. On MI350X at its default 1000 W package limit the native FP64 GEMM is held at the limit 82% of the time at 1890 MHz; with 0 kept bits it draws 785 W at 2183 MHz and is 17% faster.

Toggling or zero? Zeroing stops the low bits from varying and makes them zero. Filling them with ones or a fixed nonzero pattern instead also stops them toggling but is not zero. On GH200 we compare module power at 1000 W or throughput at 500 W; on MI350X we lock the maximum graphics clock at 900, 1300 or 1700 MHz, below the frequency any configuration reaches, so that none is power-limited (PPT residency 0–0.6%) and the energy per operation compares switching activity directly (8 GPUs × ABBA).

Most of the benefit survives such fills, in matrix engines, FP64 vector units and memory copies of both vendors (Table II; all kernels in Supplement Table S12), supporting the toggling hypothesis [2] beyond one vendor or unit. On MI350X the shares barely change with the clock (FP64, 27 bits: 81–83% at 900–1700 MHz), while the total saving grows with it (FP64, 0 bits: 20, 24 and 29% less energy per operation), as expected when CV^2f grows relative to static power. The effect is not limited to matrix engines: an FP64 multiply-add loop with 0 kept bits draws 553 instead of 862 W on GH200 at 1000 W and runs 33% faster on MI350X at 600 W, and a memory copy, power-bound on MI350X at 600 W, runs 28% faster.

Bit changes, not values. Fills also change values. Three further controls on MI350X keep values or set bits fixed and vary only how bits change between the operands that meet in the matrix engine (Table III). Placing the same values per row and column in sorted instead of shuffled order along k lowers the energy per operation by 8–9 points with 52 kept bits and by 8–9 points with 0, with identical values and set bits. Random dropped bits that differ between rows but repeat along k (krep) save almost nothing, whereas the transpose, random

TABLE II

SHARE OF THE ZERO-FILL BENEFIT (%) RETAINED WHEN THE DROPPED BITS ARE FILLED WITH ONES / ONE FIXED PATTERN INSTEAD OF ZEROS: MODULE POWER AT 1000 W (GH200), ENERGY PER OPERATION AT LOCKED CLOCKS OR THROUGHPUT AT 600 W (MI350X), 0 KEPT BITS.

Kernel	GH200	MI350X
FP64 GEMM	76 / 68	73–75 / 69
BF16 GEMM	92 / 92	77–78 / 68–72
FP64 multiply-add loop	87 / 62	74 / 63
Memory copy	97 / 86	83 / 41

TABLE III

MI350X, FP64 GEMM AT LOCKED CLOCKS: BIT FLIPS BETWEEN NEIGHBOURING OPERANDS ALONG k AND ACROSS ROWS OF A / COLUMNS OF B, SET BITS PER ELEMENT, AND ENERGY PER OPERATION RELATIVE TO NATIVE AT 1300 / 1700 MHz (8 GPUs × ABBA; 95% INTERVALS WITHIN ±0.5 POINTS). SORTED: THE SAME VALUES PER ROW AND COLUMN AS SHUFFLED, REORDERED ALONG k. KREP / MREP: RANDOM DROPPED BITS THAT REPEAT ALONG k / ACROSS ROWS. SEVEN OF THE 15 LAYOUTS; ALL 15 AND THE N02 RUN IN SUPPLEMENT S9.

Bits	Layout	Flips k	Flips row	Set bits	Energy/op (%)
52	shuffled	27.2	27.2	34.5	+0.6 / +0.7
52	sorted	23.2	21.2	34.5	−7.3 / −8.6
27	zeros	15.2	15.2	22.8	−8.4 / −9.6
27	krep	15.2	27.7	35.3	−0.6 / −0.5
27	mrep	27.6	15.2	35.3	−3.7 / −4.2
0	zeros	1.7	1.7	9.3	−24.5 / −28.6
0	sorted	0.0	0.0	9.3	−32.0 / −38.1

along k but shared by all rows (mrep), saves about half as much as zeros: the engine sees operands from neighbouring rows as well as neighbouring k in succession, and not with equal weight. We fit the energy change ΔE (%) of the 15 layouts on the changes from native of the flips per neighbouring pair along k (ΔF_k: consecutive k in a row of A or column of B) and across rows (ΔF_r: neighbouring rows of A, columns of B), each the mean over A and B of per-bit-position counts of the measured operands, and score each model by predicting every layout from the other 14 (leave-one-layout-out R^2, three runs). The total number of flips gives 0.79 and set bits 0.34. $\Delta E = a\,\Delta F_k + b\,\Delta F_r + c\,\Delta F_k \Delta F_r$ without intercept gives 0.93 (largest error 5.8 points), with $a = 0.062$, $b = 0.43$, $c = -0.021$ at 1300 MHz on n01: near native a flip across rows costs seven times one along k, and the negative interaction makes a bit save most when it is quiet in both directions, which is why krep and mrep do not add up to zeros. Weighting flips by bit significance fails ($R^2 < 0$), and separating sign and exponent from mantissa bits adds nothing. The asymmetry suggests that the operand path sees row neighbours more often than k neighbours, as when one operand is broadcast across lanes; we cannot observe the dataflow. These controls cover FP64 on MI350X only.

A second clock effect on MI350X. With the clock locked and nothing power-limited, the effective clock still depends on the data: at a 1300 MHz lock the native FP64 GEMM runs steadily at 1171–1179 MHz (5th–95th percentile, 8 GPUs) and with 0 kept bits at 1235–1240 MHz; native BF16 at 1088–1100 MHz and with 0 bits at 1132–1142 MHz, at 59–63 °C, while power and thermal residency and the per-XCD "clock below host limit" counters read zero. Circuit-level clock stretching or droop protection would explain it; we have no direct measurement. Truncation relieves it too.

V. A Model of Truncation Gains Under a Power Cap

Let a GPU under cap P settle at a target clock f. Its voltage $V(f)$ is set by the target clock, but switching happens at the effective clock sf, where $s \le 1$ is the data-dependent clock reduction of Section IV ($s = 1$ when there is none). With c the program's effective switched capacitance and P_0 the power that does not scale with switching,

$$P = P_0 + c\,s\,V(f)^2 f = P_0 + c\,s\,g(f), \tag{1}$$

and performance is proportional to sf. Truncation changes c and s, not g. For the native (n) and truncated (r) program define

$$R = \frac{c_r s_r}{c_n s_n}, \qquad \sigma = \frac{s_r}{s_n}; \tag{2}$$

at a common locked target clock, R is the ratio of their dynamic powers and σ the ratio of their effective clocks, both directly measurable. Under cap P the truncated program settles where $c_r s_r g(f_r) = P - P_0$; the native program reaches the same target clock f_r under P' with $c_n s_n g(f_r) = P' - P_0$. Dividing,

$$P' = P_0 + \frac{P - P_0}{R}, \qquad p_r(P) = \sigma\, p_n(P'), \tag{3}$$

$$\Delta W_f = P' - P = (P - P_0)\left(\tfrac{1}{R} - 1\right), \tag{4}$$

where $p_r(P) = \sigma\, p_n(P')$ because both run at target clock f_r and performance scales with s. ΔW_f is the extra cap the native program needs to reach the truncated program's *target clock*. The equivalent saving used in our results, the extra cap it needs to reach the truncated program's *throughput*, is

$$\Delta W = p_n^{-1}(\sigma\, p_n(P')) - P, \tag{5}$$

which equals ΔW_f only when $\sigma = 1$ (GH200, by assumption) and exceeds it when $\sigma > 1$ (MI350X, $\sigma = 1.01$–1.05). Every ΔW reported in this paper is measured, by interpolation along the measured native curve; the model is used only to predict speedups. The model assumes that s does not change with f over the range used and that the program stays compute-bound; predictions are capped at the program's clock-limited peak. The model needs the native curve $p_n(P)$ and P_0; it saves measuring every truncated variant under every cap. We set P_0 to the measured idle power, independent of the validation data: 174 W for a GH200 module (three nodes 169–179 W) and 262 W per MI350X GPU. The truncated kernel is characterized by locked-clock runs at 1300 MHz (MI350X; R, σ; no point of the cap sweep) or by its speed at the lowest cap $P_{\min} = 300$ W (GH200, which we could not clock-lock; $\sigma = 1$, so Eq. (3) inverts to $R = (P_{\min} - P_0)/(p_n^{-1}(p_r(P_{\min})) - P_0)$; that point is excluded from evaluation), and predictions are compared with the GEMM sweeps at the other caps (400–1000 W, seven caps per format on both platforms).

The mean absolute error is 1.5 percentage points on GH200 (largest 4.6) and 2.1 on MI350X (largest 10.4; Fig. 1). Fitting P_0 on all other formats and predicting the held-out one gives

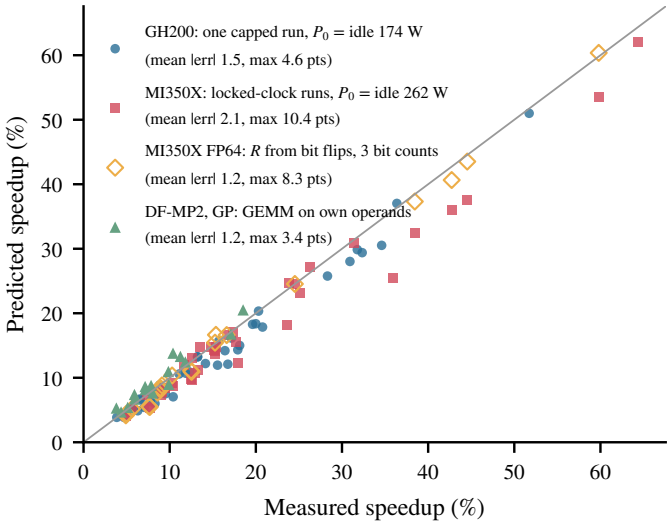


Fig. 1. Predicted against measured GEMM speedups under caps at 400–1000 W (GH200: FP64 with 0, 12, 27 and 35 kept bits, BF16 with 0 and 3; MI350X: FP64 with 0, 27 and 36, BF16 with 0 and 3, FP8 with 0), with P_0 set to the measured idle power; circles: GH200, calibrated at its lowest cap; squares: MI350X, calibrated by locked-clock runs of each truncated variant; open diamonds: R predicted from bit flips; triangles: the two applications at 500–1000 W, calibrated by a locked-clock GEMM on their own operands.

1.0 and 2.3; treating all power as dynamic ($P_0 = 0$) gives 6.1 and 4.0, and omitting σ on MI350X 2.5. The ratios are $R = 0.94$, 0.89 and 0.67 for FP64 with 36, 27 and 0 kept bits, 0.92 and 0.85 for BF16 with 3 and 0, 0.90 for FP8 with 0, and $\sigma = 1.01$–1.05 on MI350X; on GH200 $R = 0.88$, 0.82, 0.69 and 0.54 for FP64 with 35, 27, 12 and 0 bits and 0.87 and 0.77 for BF16 with 3 and 0. For the FP64 widths applications keep (27–36 bits) the error is at most 2.9 points; it is largest where the truncated clock lies beyond the native curve. **Baselines.** Carrying the speedup measured at one cap to the others errs by 19 and 10 points on average (GH200, MI350X; largest 73 and 48), the locked-clock energy ratio or clock ratio alone by 6.4 and 15.9 respectively (MI350X), and linear interpolation between the lowest and highest cap by 6.0 and 1.8, against 1.7 and 2.0 for the model on the same points: on MI350X the model is not more accurate, but it needs no capped run, interpolation two. A fitted P_0 (190–250 W) lies below the idle power, absorbing the shape error of $g(f)$, so we use the idle value. **From bit flips.** R and σ can also be predicted without running the truncated variant: fitting them, at the 1300 MHz lock, on the flip changes of the 12 layouts that are not plain truncation (Supplement Table S7) and evaluating the fit on the flips of truncated operands gives $R = 0.935$, 0.877 and 0.619 for 36, 27 and 0 kept bits (measured 0.934, 0.887, 0.654), and the resulting FP64 GEMM speedups under caps err by 1.2 points on average (largest 8.3) over 21 points, against 2.3 from the locked-clock calibration (open diamonds in Fig. 1). The 21 points are three bit counts at seven caps, so they test only three independent predictions of R and σ, all from synthetic operands. For a whole application this GEMM-derived fit is not enough: applied to the flips of DF-MP2's tensor it predicts $R = 0.90$ and 0.81 for 32 and 20 bits (measured 0.92, 0.86) and overpredicts the gains by 4.4 points on average (largest 9.0). The cause is the kernel, not the data: at DF-MP2's shape ($2002 \times 302302 \times 5477$, A transposed) synthetic operands give $R = 0.915$ and 0.848, within 0.005 of DF-MP2's own operands, so the per-flip energies of the fit belong to the kernel it was fitted on and must be refitted per shape. **External validation.** The GEMM points were used to develop the model. As a test on programs that played no part in it, we ran DF-MP2 and the Gaussian-process likelihood at the 1300 MHz lock on node n02, derived R and σ from their whole-program energy and time ($R = 0.92$ and 0.86, $\sigma = 1.02$ and 1.03 for DF-MP2 with 32 and 20 bits; $R = 0.94$ and 0.91, $\sigma = 1.01$ and 1.02 for the likelihood with 36 and 28 bits), and predicted their speedups at 500–1000 W from their native curves on the same node. Over the 16 points the mean error is 0.8 percentage points and the largest 2.9. The sampler and the energy counter read inside the program agree within 0.001 in R here, as on the GEMM (0.0007). DF-MP2 reproduces an earlier run within 0.001; two earlier runs of the likelihood gave $R = 0.93/0.89$ and 0.96/0.93 (errors 1.1 and 2.3; 1.1 and 1.7 over all 16 points): in one, its unchanged non-GEMM phases drew 3–7% less power in the truncated variants, in the other these ran 7 °C hotter. At a locked clock, leakage moves R with temperature; we report the run whose temperatures and non-GEMM power match. **From the application's operands.** The truncated program need not run at all: the GEMM microbenchmark at the lock on the application's own operands (DF-MP2's flop-weighted mean call, with calls 50 and 225 within 0.004 in R; four GP trailing-update panels, within 0.003) gives $R = 0.910$ and 0.844 (DF-MP2, 32 and 20 bits) and 0.927 and 0.886 (likelihood, 36 and 28). With φ the GEMM's share of the native dynamic energy and the other phases unchanged, $R_{\text{app}} = 1 - \varphi(1 - R)$ and $\sigma_{\text{app}} = 1 + \varphi(\sigma - 1)$, a first-order form that treats the run as one phase. In the likelihood, read at phase boundaries, the other phases (POTRF, TRSM, covariance, solve) draw 363 against 650 W, so $\varphi = 0.97$ although the GEMM takes 90% of the time, and the GEMM's in-program R (0.885–0.907, 0.930–0.940 in two runs) brackets the panel's; in DF-MP2 the GEMM takes 96% of the time and 95% of the dynamic energy. This predicts the 16 speedups (four R values at four caps) within 1.2 points on average (largest 3.4; without φ, 1.5), within the 0.8–1.7 of the whole-program runs. A truncated program's curve thus follows from one locked-clock GEMM on its operands to within a few points. Errors of 1–2 points are small against gains of 5–20 points, not against the margins near a crossover.

VI. Results: Gains Under Power Caps

All results below truncate inside the program and compare with the unmodified program (on MI350X, DF-MP2 and the Gaussian process are our own ports). Fixed bit counts in Quantum ESPRESSO, GPU4PySCF and ExaGeoStat were also measured through a GH200 interposer that copies the operands; those numbers isolate truncation but pay a 3–9% copy cost, are slower than the unmodified program in two of

three cases, and serve only as upper bounds (Supplement S1). Three third-party FP64 codes whose stored coefficients we truncated (MFEM, SW4, LAMMPS) gain 0–2%, because the truncated data are a small share of their traffic (Supplement S2).

A. GEMM

Zeroing speeds up GEMM at every cap on both platforms, though not monotonically as the cap tightens (Fig. 2). On GH200, FP64 with 27 bits is 8.1–11.1% faster at 500–800 W (ΔW 79–162 W) and BF16 with 0 bits 11.9–28.3%. On MI350X the default limit already binds: at 1000 W FP64 with 36, 27 and 0 kept bits is 5.1, 9.2 and 16.9% faster, BF16 with 3 and 0 bits 7.7 and 15.6% and FP8 with 0 bits 8.8%, and at 500 W 8.6, 15.2, 60.0, 13.5, 30.6 and 17.4% (4 windows on two nodes, intervals $\pm$0.1–1.6; nodes within 1.8 points).

B. HPL and HPL-MxP

The residual check determines the usable bit count, must be checked at several sizes, and depends on the platform (Supplement Table S8). On GH200 27 bits sits within one bit of the threshold; on MI350X it fails at every size, by 6–32×, and so does 29 bits. 35 bits passes on both with a margin of 35–100×; the platforms differ in HPL build and DGEMM blocking and accumulation order, so a bit count validated on one does not transfer.

GH200, 8 nodes. Tuned native HPL reaches 393.9 TFLOP/s at 1000 W. With 35 bits it is 3.3–4.7% faster at 500–800 W over 6 pairs from two node sets (ΔW = 42–74 W per node), and at the default cap mainly saves energy (7.5–8.6% per solve); node sets differ by up to 0.9 points (Supplement S6). HPL-MxP with 3 kept bits is 1.7–5.8% faster at 500–800 W; refinement grows from 1 to 3 iterations, eating most of the LU's 5–16% speedup above 600 W; at 1000 W native HPL-MxP trips the chassis power brake at full fan speed, so we exclude that cap.

MI350X, one node of 8 GPUs. Native HPL reaches 424.9 and 429.8 TFLOP/s on the two nodes at 1000 W. With 35 bits it is 4.6–5.1% faster at the default limit, 4.5–4.9% at 800 W, 6.2–8.0% at 650 W and 6.3–7.3% at 500 W (Table IV), with 4.6–6.8% less energy per solve and ΔW of 39–50 W per GPU at 650–800 W (about 0.35–0.4 kW per node). HPL-MxP, 7.61 and 7.65 PFLOP/s natively, reaches 7.98 and 8.04 PFLOP/s with 3 kept bits at the default limit (+4.9 and +5.1%) and gains 9.2–19.5% at 500–800 W, with 5.3–14.6% less energy per solve. Refinement grows from 2 to 4 steps (about 1.5% of the LU time). Keeping 0 bits does not converge on MI350X, so 3 bits is the operating point on both platforms. Apart from the 800 W cell discussed in Section III, temperature adjustment moves no HPL or HPL-MxP value by more than 0.7 points. For GP and DF-MP2, whose truncated variants ran in the middle, hottest part of each round, adjusting to equal starting temperature raises the gains by 0.6–2.4 points; the controlled check of Section III (DF-MP2 12.6 $\pm$ 0.1%, likelihood 7.3 $\pm$ 0.1) lies within the range of the thermal methods, and Tables V and VI give the start-adjusted values at the other caps. **A harder matrix.** These values use the code's diagonally dominant test matrix (diagonal equal to the absolute off-diagonal row sum, about $n/2$). With the diagonal set to $\sqrt{n}$ instead [29], far from dominance, and exact diagonal-tile factorization, native HPL-MxP (N = 1034752) passes after one Richardson and three GMRES steps at 6.97 PFLOP/s; 6 to 3 kept bits all pass (3 bits with one more step), and 3 bits is 5.0 and 16.3% faster at 1000 and 500 W. The generator is ours, not an official one. On two nodes (16 GPUs, N = 1453056, 4 runs per variant) 3 bits is 6.0% (5.9–6.1) faster at 1000 W and 16.5% (12.7–20.3) at 500 W, with residual 11.8 of 16. The solver stops at the first iterate below 16, so the step count, not the residual, tracks the LU's accuracy: with equal steps the residual falls with more bits and larger N (one node, 3/4 bits: 3.4–4.2/1.5–1.8 at N = 264192, 0.59–0.70/0.30–0.34 at 792576), while on 16 GPUs at N = 1034752 4 bits passes after the Richardson step alone (5.7) and 3 bits after two more GMRES steps (0.20). Safe bit counts must be validated per configuration.

C. Quantum ESPRESSO: Tolerance-Driven Bit Counts (GH200)

Plane-wave DFT tolerates 20–28 kept bits; Gaussian-basis DFT needs 32–36 (Supplement S1). Tying the bit count to the Davidson threshold, in-program, reproduces the native total energy in every printed digit (10^{-8} Ry) on both systems with the same SCF iteration count, while cutting time by 2.1–3.6% and energy by 2.2–3.4% against the unmodified program (Supplement, Table S6); b rises from 16 to 39–42 as the SCF converges. On Si 1000 the work is unchanged (177–178 Davidson iterations), so its gain is a power effect. On the gold surface fixed 24 bits keep the native 138 Davidson iterations and shorten the SCF by 3.2%, the power effect at equal work; the tolerance-driven rule needs 126 (9% fewer, for reasons not yet identified) and shortens it by 4.5%, so its extra gain comes from the iteration count. The rule needs no safe bit count in advance and matches or beats the best fixed count.

D. A Power-Hungry Application: DF-MP2

The three-index tensor B is zeroed in place once; the contraction $B_i B_j^{\mathsf{T}}$ (FP64 DGEMM with the auxiliary basis as inner dimension, k = 5477) dominates the run. Keeping 32 bits changes the MP2 correlation energy by at most 5×10^{-12} E_h and 20 bits by at most 2.2×10^{-8} E_h (Table V). On GH200 the truncated variants run at native speed at 1000 W with 8–14% less energy and gain 5–8% under caps; on MI350X, held at the limit 95% of the time at 1000 W, they are 5.7 and 10.3% faster there and 9.8 and 18.6% at 500 W (4 rounds on two nodes). Emulation behaves differently on the two platforms (Section VII-A).

E. A Wide-Range Workload: Gaussian-Process Likelihood

The likelihood factors an $n\times n$ Matérn covariance by Cholesky; each panel L_{21} keeps b bits in one fused, coalesced pass before its trailing update. The panel rows span 162 binades, and 88% of their nonzeros lie more than 2^{-32} below their row maximum. With 28 bits the log-likelihood

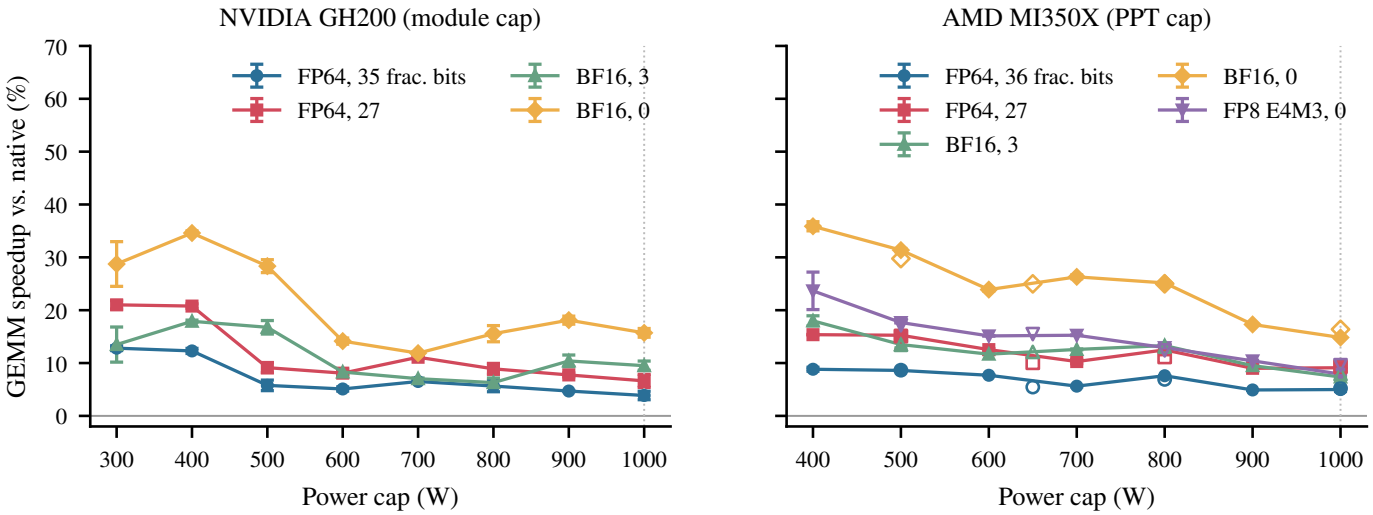


Fig. 2. GEMM speedup from mantissa zeroing under power caps (GH200 keeps 35 fraction bits, as HPL; MI350X 36). GH200 (module cap; FP64 $m = n = 32768$, $k = 2048$, BF16 $k = 6144$): 95% t intervals over 6 node–pair samples (3 nodes × 2 ABBA pairs). MI350X (package cap; BF16 and FP8 $k = 8192$): mean of node n02's two measurement windows, each averaging the 8 GPUs that run at once; bars span the two windows; open markers: node n01. The dotted line marks each platform's default limit, which is also its highest.

TABLE IV

MI350X: SPEEDUP (%) OVER THE UNMODIFIED PROGRAM AT THE DEFAULT LIMIT AND AT 500 W. POOLED: MEAN ± 95% t INTERVAL OVER THE ANALYSIS UNITS OF BOTH NODES (WINDOW: ONE MEASUREMENT WINDOW WITH THE 8 GPUS AVERAGED; RUN: ONE NODE-WIDE RUN; ROUND: ONE ROUND OF 8 GPUS). PER NODE: RAW (SAME COOLING ENVIRONMENT) / ADJUSTED TO EQUAL JUNCTION TEMPERATURE AT THE START OF EACH RUN (SECTION III). THE INTERVALS ARE STATISTICAL ONLY: ACROSS THERMAL METHODS DF-MP2 (20 BITS) AND THE LIKELIHOOD SPAN 10.3–14.2 AND 7.1–8.2% AT 1000 W.

		1000 W			500 W		
Result	Unit, n	Pooled	n01 raw/adj.	n02 raw/adj.	Pooled	n01 raw/adj.	n02 raw/adj.
GEMM FP64, 36 bits	window, 4	5.1 ± 0.6	5.2/5.4	5.0/5.5	8.6 ± 0.1	8.6/8.6	8.6/8.8
HPL, 35 bits	run, 6	4.8 ± 0.3	4.6/4.5	5.1/4.8	7.0 ± 0.6	7.3/7.9	6.3/6.5
HPL-MxP, 3 bits	run, 8	5.0 ± 0.4	4.9/4.4	5.1/5.2	19.3 ± 1.3	19.5/19.0	19.1/20.5
GP likelihood, 28 bits	round, 4	7.4 ± 0.6	7.6/8.2	7.1/8.0	11.0 ± 1.0	10.8/12.5	11.2/12.9
DF-MP2, 20 bits	round, 4	10.3 ± 0.6	10.2/11.5	10.4/11.8	18.6 ± 0.3	18.8/19.9	18.5/20.1
DF-MP2, 32 bits	round, 4	5.7 ± 0.4	5.5/6.6	5.9/6.7	9.8 ± 0.1	9.7/10.5	9.8/11.1

moves by a relative 3×10^{-8}–4.5×10^{-8}, far below what matters for estimation, a one-parameter maximum-likelihood fit on GH200 returns the native estimate in every printed digit (Supplement S5), and a full three-parameter fit on MI350X (variance profiled, range and nugget by Nelder–Mead, two data sets) moves the estimates by at most 0.25% with 28 bits and 0.004% with 36, against a 2% difference in range between the two data sets. On GH200 the gain at 1000 W depends on how often native is capped (3.8% here, 0.1% with a Morton ordering that is capped less); on MI350X the limit binds and 28 bits is 7.4 ± 0.6% faster at 1000 W and 11.0 ± 1.0% at 500 W (4 rounds on two nodes). FP32 is unusable for estimation; truncating the FP64 tiles that remain in a tile mixed-precision Cholesky makes it a further 1.9–7.0% faster, so truncation composes with format changes (Supplement S5).

VII. THE BOUNDARY WITH INT8 EMULATION OF FP64

Emulation wins by a wide margin where its fixed costs amortize at a generous cap (long inner dimensions: HPL, DF-MP2, GEMM with $k \geq 2048$); a tighter cap or accuracy threshold shrinks or reverses its lead, and it loses where the inner dimension is short (the likelihood, GEMM with $k = 1024$). Truncation gains less but in every case we measured, needs no extra memory, and alone applies to BF16 and FP8.

A. Crossover: Shape, Cap and Memory

Emulation trades FP64 arithmetic for several times as many integer operations, so its benefit grows with the ratio of INT8 to FP64 throughput, about 30 on GH200 and about twice that on MI350X. On GH200, cuBLAS emulation at the accuracy of 35-bit truncation ($\approx 10^{-11}$) is 11–28% slower than truncation on a square GEMM with $k = 2048$, and wins only for long inner dimensions; across 480 measurements a three-parameter model in $1/m + 1/n$ and $1/k$ describes GEMMul8 with $R^2 = 0.98$–0.99 (Supplement S4). On MI350X the crossover lies further toward emulation, and the cap moves it (Fig. 3). Truncation to 35 bits (7.8×10^{-12}) is 5–6% faster than native at 1000 W and 8–9% at 500 W for every k from 1024 to 8192, whereas GEMMul8 with 12 moduli ($\approx 5\times10^{-13}$) goes from −13 to +121% at 1000 W and from −45 to +48% at 500 W. Fig. 3 compares the two at equal accuracy, sweeping the kept

TABLE V

DF-MP2 (TAXOL DEF2-TZVP): SPEED RELATIVE TO NATIVE (MEAN ± 95% t INTERVAL; GH200: 6 ROUNDS FROM TWO JOBS ON TWO NODES; MI350X: 4 ROUNDS OF 8 GPUS, TWO PER NODE ON TWO NODES), AND THE ERROR $|\Delta E|$ OF THE MP2 CORRELATION ENERGY (TRUNCATED VARIANTS ROUND TO NEAREST). MI350X GEMMUL8 TIMES EXCLUDE THE ONE-TIME SETUP OF THE HOOK (SECTION VII-A). GEMMUL8 WAS NOT RUN FOR DF-MP2 ON GH200, SO THE EMULATION COLUMNS DIFFER IN LIBRARY AS WELL AS HARDWARE. MI350X 20 BITS AT EQUAL STARTING TEMPERATURE (N01/N02): 11.5/11.8, 14.8/12.7, 17.6/18.8 AND 19.9/20.1% AT 1000–500 W; CONTROLLED CHECK AT 1000 W: 12.6 ± 0.1%.

Cap	Native 32 bits	20 bits	Emulation
GH200 (emulation: cuBLAS, 32 bits)			
1000 W	39.5 s	+0.5 ± 0.2%	+0.8 ± 0.1%	−35%
700 W	43.9 s	+4.9 ± 0.4%	+7.4 ± 0.4%	−28%
500 W	51.5 s	+4.7 ± 0.2%	+7.7 ± 0.5%	−18%
MI350X (emulation: GEMMul8, 12 moduli)			
1000 W	21.7 s	+5.7 ± 0.4%	+10.3 ± 0.6%	+66.6 ± 1.1%
800 W	26.4 s	+6.7 ± 1.4%	+12.7 ± 1.8%	+60.7 ± 0.9%
650 W	34.1 s	+9.5 ± 0.7%	+16.6 ± 0.9%	+44.8 ± 1.4%
500 W	55.8 s	+9.8 ± 0.1%	+18.6 ± 0.3%	+32.6 ± 3.1%
$\lvert\Delta E\rvert$	—	$\le 5\times10^{-12}$	$\le 2.2\times10^{-8}$	G: 2.2×10^{-8}; M: 7×10^{-12}

TABLE VI

GAUSSIAN-PROCESS LIKELIHOOD (n = 80000, SIX RANGE PARAMETERS): LARGEST LOG-LIKELIHOOD DEVIATION FROM NATIVE AND SPEED RELATIVE TO NATIVE (MEAN ± 95% t INTERVAL; GH200: 6 ROUNDS; MI350X: 4 ROUNDS OF 8 GPUS ON TWO NODES; TRUNCATION ROUNDS TO NEAREST). THE LOG-LIKELIHOODS ARE 1.2–1.7×10^5. [a] AT n = 80000 GEMMUL8 RETURNS SUCCESS BUT AN INVALID TRAILING UPDATE; AT n = 64000 IT IS 11–43% SLOWER THAN NATIVE (SECTION VII-A). MI350X 28 BITS AT EQUAL STARTING TEMPERATURE (N01/N02): 8.2/8.0, 7.1/9.5, 9.6/10.3 AND 12.5/12.9% AT 1000–500 W; CONTROLLED CHECK AT 1000 W: 7.3 ± 0.1%.

	Trunc. 36	Trunc. 28	Emulation	FP32
GH200 (emulation: cuBLAS, 36 bits)				
max $\lvert\Delta\ell\rvert$	2.4×10^{-5}	7.2×10^{-4}	2.2×10^{-3}	87
1000 W	+3.0 ± 0.3%	+3.8 ± 0.4%	−41%	−8.2%
700 W	+5.0 ± 0.1%	+7.9 ± 0.2%	−31%	+3.2%
500 W	+4.8 ± 0.1%	+7.5 ± 0.1%	−21%	+3.9%
MI350X (emulation: GEMMul8)				
max $\lvert\Delta\ell\rvert$	1.9×10^{-5}	5.3×10^{-3}	wrong[a]	—
1000 W	+3.9 ± 0.2%	+7.4 ± 0.6%	—	—
800 W	+4.1 ± 0.6%	+7.1 ± 1.8%	—	—
650 W	+5.3 ± 1.7%	+7.8 ± 2.8%	—	—
500 W	+6.9 ± 0.7%	+11.0 ± 1.0%	—	—

bits (20–44) and the moduli (8–14). With k = 4096 emulation is faster at every accuracy except the tightest at 500 W; with k = 2048 it is faster at every accuracy at 1000 W but at 500 W only for errors of 10^{-8} or more. With k = 1024 the order depends on the accuracy and the cap: at 1000 W emulation wins for errors above about 10^{-10} (8 moduli: +27% at 1.3×10^{-8}, against +10% for 24-bit truncation) and truncation below (+4.8 against −6.2% near 5×10^{-12}); at 500 W truncation wins at every accuracy. Emulation draws the full limit, so under a cap its extra integer work is paid for with clock, whereas truncation removes work from the power budget; on GH200 the same holds for GEMMul8, while cuBLAS emulation, which draws only about 706 W at 1000 W, gains ground at lower caps: the direction depends on the emulation's own power.

Applications. In DF-MP2 on MI350X (k = 5477) GEMMul8 with 12 moduli is 34–67% faster than native and beats truncation at every cap; at the accuracy of 20-bit truncation (2.2×10^{-8} E_h) it needs only 9 moduli (2.4×10^{-8} E_h) and is 112, 86 and 65% faster at 1000, 650 and 500 W, against 12–20% for truncation (GEMMul8 3.5.4, direct API); on GH200 cuBLAS emulation is 18–35% slower than native there (Table V). In FP64 HPL on MI350X we fix a common threshold on the scaled residual and let each method use its fastest configuration that meets it, among those we measured (Supplement Table S13). HPL's own check is 16. Tighter thresholds are a sensitivity analysis, not a requirement of HPL or of any application. Under HPL's check, GEMMul8 with 10 moduli (residual 1.79; N = 491520, NB 4096, 4 × 2) leads at every cap: 576.8 TFLOP/s at 1000 W, 34% above the fastest native configuration on the same node, against 6.6% for 30-bit truncation (residual 6.4), and 11.3 against 9.4% at 500 W. A threshold of 1 excludes 10 moduli; 12 moduli (0.033) fits only up to N = 458752 and is 16.9% faster than the fastest native at 1000 W but 4.0% slower at 500 W, where 35-bit truncation gains 7.3%, a clear reversal; at 650 W, paired in one job, 12 moduli and 35 bits gain 8.0 and 6.7%, a difference (1.2 ± 2.4 points) we cannot resolve. At 0.1, 36 fraction bits (HPL setting 37) pass at the best native configuration (0.087) and gains 4.4–6.4%, leading at 500 W and close to emulation (7.8%) at 650 W. Tightening the threshold thus changes the measured order at the tightest caps.

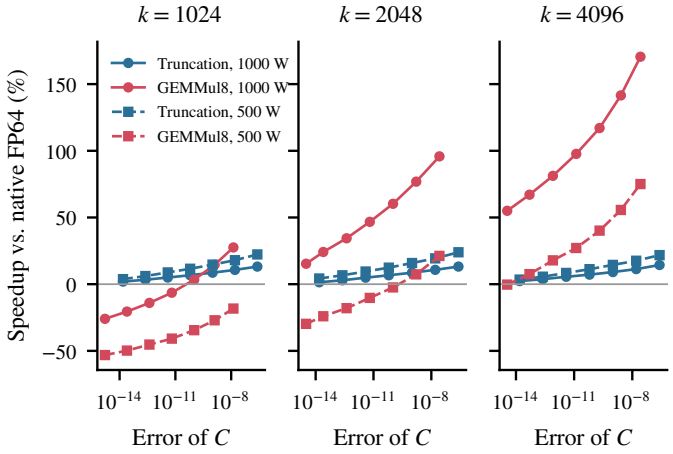


Fig. 3. MI350X, iso-accuracy comparison: speedup over native FP64 against the relative error of C for truncation (round to nearest, 44 down to 20 kept bits) and GEMMul8 3.2.0 (8 to 14 moduli), $m = n$ = 32768, at 1000 and 500 W (node n02, 8 GPUs × ABBA).

In the Gaussian-process likelihood, whose trailing updates have k = 1024, GEMMul8 is slower than native at every cap (n = 64000, where it is correct: −11, −22, −37 and −43% at 1000, 800, 650 and 500 W with 12 moduli, log-likelihood within 3×10^{-7}), while 28-bit truncation gains 5.9–11.8%.

Costs that appear only in applications. Emulation needs workspace: at our best native HPL configuration (N = 540672) the matrix fills 272 of each GPU's 288 GiB and GEMMul8's workspace no longer fits, so HPL must give up problem size first. Through the BLAS hook each new GEMM shape costs

a one-time setup (DF-MP2's first pass over its 226 shapes: 138–141 s instead of 13 s, in GEMMul8 3.2.0 and 3.5.4 alike; the times above exclude it), which calling the library directly with one workspace removes. In the likelihood at $n = 80000$ the hooked DGEMM of the first trailing update ($m = n = 78976$, $k = 1024$) returned success but an invalid result in all 256 runs: GEMMul8 batches its moduli's INT8 products with output stride mn, and ROCm 7.2.3's strided-batched INT8 GEMM silently leaves every batch after the first unwritten once the stride reaches 2^{32} elements, independent of the data, a deployment risk of replacing a BLAS call wholesale.

B. Exponent Span: When Emulation Silently Fails

Truncation keeps every element's exponent; Ozaki-type emulation slices each row relative to its maximum, so elements far below the maximum lose bits. On data with a wide per-row span this can go wrong silently (Supplement Table S14): in Gaussian-basis DFT on GH200, where 24–44% of the flop-weighted nonzeros lie more than 2^{-32} below their row maximum, fixed 36-bit cuBLAS emulation lets the SCF converge normally to a total energy 52.5 $\mathrm{mE_h}$ off, and GEMMul8 diverges, whereas rounding to 36 bits errs by 9.6×10^{-7} $\mathrm{E_h}$. With 40 bits or the automatic mode the emulation is correct but 1.5–2.1× slower than truncation or native. Uniform synthetic spreads over up to 64 (MI350X) or 256 (GH200) binades barely affect GEMMul8 (MI350X, 12 moduli: p99 error at most 4.4×10^{-11}), so a wide per-row span, which an interposer can measure beforehand (Supplement Fig. S1), is necessary but not sufficient for this failure; narrow-span codes (HPL, Quantum ESPRESSO, DF-MP2) do not suffer it. It does not flag defects such as the size overflow of Section VII-A; a check of the result does. Replacing every FP64 GEMM with FP32 or 3xTF32 fails for a different reason, accumulation over long inner products (Quantum ESPRESSO, Supplement S3).

A post-hoc spot check tests the result instead: after emulating, recompute 64 random rows (or columns, along the larger dimension) of C natively and compare normwise. It catches gross errors: at the size failure ($m = n = 65536$) the error is 32 and the estimate 31.6, and on a synthetic pair whose product terms all lie 2^{-40} below the row maxima the estimate matches the full error (100, 25 and 0.1% with 10, 12, 14 moduli). It misses the wide-span loss: on the exchange–correlation operands of the valinomycin DFT, rebuilt on the CPU (DF-B3LYP/def2-TZVP, energy within 10^{-10} $\mathrm{E_h}$ of GH200's), GEMMul8's normwise error (10^{-10} to 10^{-12}) is that of 28–36-bit truncation, but its componentwise error $\max|\Delta C|/(|A||B|)$ reaches 1, with 19–62% of the entries off by more than 10^{-6} (36-bit truncation: none above 10^{-9}). Computed on the same 64 rows (one more DGEMM, on $|A|$ and $|B|$, ignoring entries whose bound $|A||B|$ underflows below 10^{-290}), the componentwise error detects it: 1.0 on the DFT exchange–correlation operands against 10^{-11} for truncation. GP panels lose entries the same way (componentwise 1.0) without any effect on the likelihood, so the product alone cannot tell harmful from harmless loss, and a converging SCF (Supplement Table S14) is no check either. The componentwise check costs 0.6–0.7% of the emulated call on the DF-MP2 and GP shapes but more than the call on the small DFT GEMMs ($m = n \le 2958$), so it should run on the first calls of each shape, not every call.

TABLE VII

PER-CALL SELECTOR AGAINST FIXED CHOICES: SHARE OF CASES IN WHICH EACH PICKS THE MEASURED FASTEST METHOD THAT MEETS THE TOLERANCE (MEAN TIME LOST AGAINST IT). GEMM: LEAVE ONE SHAPE OUT, MI350X 22 SHAPES × 3 CAPS × 2 TOLERANCES (10^{-10}, 10^{-12}), GH200 20 SHAPES × 3 CAPS (CUBLAS EMULATION WITH 32 BITS AT 10^{-11}, THE FASTER OF IT AND GEMMUL8 WITH 10 MODULI AT 10^{-9}). DF-MP2: EVERY CALL AT 3 CAPS (12 MODULI VERSUS 20 BITS). GP: WHOLE RUNS, BLOCK SIZE k (12 MODULI VERSUS 35 BITS, BOTH WITHIN 10^{-11}; THE TOLERANCE CHAIN OF SECTION VII-C VARIES THEM). SPAN STATISTIC: THE SIX REAL-DATA CASES OF SUPPLEMENT FIG. S1; SPOT CHECK: DETECTED AMONG THE SIZE FAILURE AND THE SYNTHETIC HIDDEN PAIR (GROSS), VALINOMYCIN XC (WIDE-SPAN, HARMFUL) AND GP PANELS (WIDE-SPAN, HARMLESS); REFERENCE RUN: VALINOMYCIN REJECTED, GP ADMITTED. GP RUNS ASSUME THE REFERENCE RUN ADMITTED EMULATION. NO APPLICATION WAS USED IN THE FIT.

Validation	Cases	Selector	Always emul.	Always trunc.
MI350X GEMM	132	97.7 (0.1)	51.5 (39)	48.5 (24)
GH200 GEMM, 10^{-9}	60	98.3 (0.1)	40.0 (72)	60.0 (24)
GH200 GEMM, 10^{-11}	60	83.3 (1.3)	20.0 (94)	80.0 (3.0)
DF-MP2 calls	678	99.3 (0.0)	99.3 (0.0)	0.7 (29)
GP runs	24	22 (0.3)	12 (16)	12 (11)
Span statistic	6	5/6	—	—
Spot (normwise), gross	2	2/2	—	—
Spot (normwise), span loss	1	0/1	—	—
Spot (compon.), loss	2	2/2	—	—
Reference run, harm	2	2/2	—	—

C. Choosing per Call

The pattern above can be turned into a per-call rule. For a GEMM call (m, n, k) under cap P with a tolerated relative error ε of the product, each method takes its cheapest setting whose measured error is at most ε (kept bits, or moduli), and the method with the smallest predicted time wins. Truncation's speedup barely depends on the shape (35 bits: 4.6–6.8, 5.0–7.4 and 8.0–9.8% over 22 shapes at 1000, 650 and 500 W), so it is one number per cap, or the model of Section V. Emulation's time relative to native is modelled as $r_e = c_0 + c_1/k + c_2\,(1/m + 1/n)$ per cap and number of moduli, fitted on 22 shapes ($m = n$ from 4096 to 32768, k from 512 to 8192, two rectangular; GEMMul8 3.5.4, $R^2 = 0.97$–0.99); calls through the hook add its one-time setup per shape. The rule uses only GEMM measurements. Emulated calls are spot-checked componentwise (threshold 10^{-6}) on the first calls of each shape; a failed check repeats the call natively and, if the error is a wide-span loss rather than a gross error, admits emulation for that application and input only after one reference run, native against emulated, meets the application's tolerance. On GH200 that run rejects valinomycin (52.5 $\mathrm{mE_h}$); on MI350X it admits the likelihood ($|\Delta\ell| \le 4.2\times10^{-7}$). A wide per-row span (more than 1% of the elements more than 2^{-32} below their row maximum) predicts which inputs need it.

On MI350X the rule picks the fastest method in 97.7% of held-out GEMM cases (Table VII); the best single threshold on k (≥ 2048) reaches 92.4% but loses up to 33%, although it is chosen in-sample while the rule is scored leave-one-shape-out.

Refitted on GH200 (cuBLAS emulation, GEMMul8), the rule is right in 98.3% of cases at 10^{-9} but only 83.3% at 10^{-11}, where truncation nearly always wins. DF-MP2 (k = 5477) does not discriminate: emulation wins 99.3% of its calls, as the rule predicts. The likelihood does, because its block size sets k (1024 to 4096) and the crossover moves with the cap, from k = 1536 at 1000 W to 4096 at 500 W (k = 2048: +21.7 against +4.1% at 1000 W, −19.8 against +8.3% at 500 W). The rule is right in 22 of 24 runs, missing two within 4% of the crossover; either fixed choice is right in 12, the best k threshold (in-sample) in 18. These 22 assume that the likelihood's reference run admitted emulation; refusing emulation on the span alone would cost up to 32% there (k = 4096, 1000 W).

From an application tolerance. The rule's tolerance is that of the product. For the likelihood a factor $\kappa = |\Delta\ell|/\varepsilon$, the larger of two earlier truncated runs (4.1×10^{6}), maps a tolerance τ on $|\Delta\ell|$ to $\varepsilon = \tau/\kappa$. For $\tau = 10^{-2}$, 10^{-4} and 10^{-6} the rule takes 28, 36 and 40 bits; a separate run (n = 64000, block 2048, 1000 and 500 W) confirms its choice in all six cases, but 40 bits gives 1.2×10^{-6} because κ varies fourfold between runs, so the mapping needs a margin of a bit or two. Emulation needs its own κ (5.2×10^{5}, ten times smaller): with it the rule takes 9, 11 and 12 moduli, all within τ, without changing its choices. κ need not come from full runs: it grows roughly with n (κ/n = 36–77 in runs of a few seconds at n = 16000 and 32000, 25–78 at 64000–80000), and $77n$ would have taken 44 instead of 40 bits at 10^{-6}.

VIII. Discussion

Where truncation pays. Three conditions decide the gain: the GPU must be power-bound, the truncated operands must flow directly and in volume into multipliers or data paths, and the algorithm must tolerate the error, ideally by correcting it. The model makes the first quantitative; the second explains why GEMM-dominated solvers gain 4–20% and codes whose intermediates regrow full mantissas (MFEM, SW4, LAMMPS) little; the third separates solvers from AI weights (Supplement S7). On GH200 the gain at the default cap is mainly energy; on MI350X, whose default limit binds for FP64, BF16 and FP8 matrix work, truncation turns into 4.6–10% more speed at stock settings; a part not power-bound at stock settings should see energy instead (not measured). Truncation is not an alternative to clock tuning: at a fixed clock it lowers the energy per operation (FP64 GEMM, 36 and 27 kept bits: 4.8 and 8.1–8.4% at 1300 MHz), so it lowers the cost of whatever operating point a cap or a frequency choice selects.

Truncation and emulation. Emulation is the stronger lever where it can amortize its fixed costs (long inner dimensions, generous caps, narrow exponent spans, ample memory, few distinct shapes); truncation elsewhere, and only truncation applies outside matrix multiplication and to BF16 and FP8. Building the selector into a BLAS dispatcher, with the componentwise spot check and per-input reference runs, remains to be done.

Limitations. (i) Two platforms, one generation each, and at most two nodes (16 GPUs) on MI350X; other parts manage power and cool differently. (ii) Safe bit counts depend on platform and problem size. (iii) HPL-MxP was run on the diagonally dominant test matrix and one non-dominant generator of ours, not an official one. (iv) Each scientific code was tested on one or two inputs; on MI350X DF-MP2 uses our own contraction of PySCF's tensor. (v) Analysis units within one or two nodes may be correlated within a job; on GH200 node sets differed more than pairs within a set. (vi) ABBA cancels linear thermal drift but not fan feedback; temperature-adjusted values were checked under controlled thermal history for GEMM, HPL-MxP, DF-MP2 and the likelihood, and fan speed could not be fixed. (vii) Default caps are also the highest; the limiters' time scales are unknown. (viii) Power comes from the devices' sensors and accumulators, not an external meter; GH200 module power includes the Grace CPU. (ix) The MI350X clock reduction attributed to clock stretching is an inference. (x) Part of the Quantum ESPRESSO gain on the gold surface comes from 9% fewer Davidson iterations, for unidentified reasons. (xi) GEMMul8's memory limits and the size failure are observations about one software stack (ROCm 7.2.3), and GEMMul8 was not run for DF-MP2 on GH200. (xii) Each emulation safeguard was validated on one or two cases. Checking only the first calls of a shape assumes the operands' span does not grow during the run (SCF densities, late Cholesky panels); re-checking when the span statistic changes would remove it. A reference run pays off only for inputs evaluated many times, such as a likelihood inside a maximum-likelihood loop. (xiii) TOP500 requires full FP64 arithmetic [45] and HPL-MxP allows diagonally dominant matrices for testing only [42]: our HPL and HPL-MxP numbers are research results.

IX. Conclusion

On GPUs of two vendors, truncating unused mantissa bits in software buys performance for HPC applications under power limits while their results stay within each code's own criterion. The saving comes from bits that change between neighbouring operands; a first-order model predicts an application's performance–cap curve from one locked-clock GEMM on its own operands, and bit flips predict the saving for the GEMM shape they were fitted on. MI350X is power-bound at its stock limit, so truncation speeds up HPL, HPL-MxP (on a diagonally dominant and a harder test matrix), a Gaussian-process likelihood and DF-MP2 by 4.6–10% at default settings and 7–19% at 500 W. INT8 emulation is faster for long inner dimensions at generous caps, truncation for short inner dimensions, tight caps and strict tolerances; a per-call selector fitted on GEMM measurements chooses between them on both platforms. A componentwise spot check detects gross emulation errors and wide-span losses alike; whether a loss matters takes one native-versus-emulated reference run per application and input, since a converging SCF can still be 52.5 $\mathrm{mE_h}$ off. Truncation also applies to BF16 and FP8; safe bit counts must be validated per platform and size. All HPL and HPL-MxP numbers are research results, not official scores.

References

[1] S. Bhalachandra, B. Austin, S. Williams, and N. J. Wright, "Understanding the impact of input entropy on FPU, CPU, and GPU power," arXiv preprint arXiv:2212.08805, 2022.

[2] T. Gregersen, P. Patel, and E. Choukse, "Input-dependent power usage in GPUs," in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1872–1877.

[3] O. Antepara *et al.*, "Benchmark-driven models for energy analysis and attribution of GPU-accelerated supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, 2025, pp. 888–904.

[4] H. He, "Strangely, matrix multiplications on GPUs run faster when given "predictable" data! [short]," Thonk From First Principles (Substack blog), https://www.thonking.ai/p/strangely-matrix-multiplications, Apr. 2024.

[5] J. Y. F. Tong, D. Nagle, and R. A. Rutenbar, "Reducing power by optimizing the necessary precision/range of floating-point arithmetic," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 8, no. 3, pp. 273–286, 2000.

[6] J. Pool, A. Lastra, and M. Singh, "Precision selection for energy-efficient pixel shaders," in *ACM SIGGRAPH Symposium on High Performance Graphics (HPG)*, 2011, pp. 159–168.

[7] D. Guenther, T. Henriksson, R. Leupers, and G. Ascheid, "Mantissa-masking for energy-efficient floating-point LTE uplink MIMO baseband processing," in *Design, Automation & Test in Europe (DATE)*, 2016, pp. 1028–1029.

[8] P. V. Argade and D. N. Peroni, "Variable precision computing system," US Patent 11,455,766 B2, Sep. 27, 2022.

[9] K. Ozaki, Y. Uchino, and T. Imamura, "Ozaki scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique," arXiv preprint arXiv:2504.08009, 2025.

[10] Y. Uchino *et al.*, "High-performance and power-efficient emulation of matrix multiplication using INT8 matrix engines," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 1824–1831.

[11] O. Antepara, L. Oliker, and S. Williams, "Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 1205–1216.

[12] J. Lucas and B. Juurlink, "ALUPower: Data dependent power consumption in GPUs," in *IEEE 24th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2016, pp. 95–104.

[13] H. Huang, W. Shao, and J. Hammond, "Accelerating density fitting with adaptive-precision and 8-bit integer on AI accelerators," arXiv preprint arXiv:2601.08077, 2026.

[14] B. Rountree, D. H. Ahn, B. R. de Supinski, D. K. Lowenthal, and M. Schulz, "Beyond DVFS: A first look at performance under a hardware-enforced power bound," in *IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum (IPDPSW)*. IEEE, 2012, pp. 947–953.

[15] Y. Inadomi *et al.*, "Analyzing and mitigating the impact of manufacturing variability in power-constrained supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*. ACM, 2015, pp. 1–12, article 78.

[16] T. Patki, D. K. Lowenthal, B. Rountree, M. Schulz, and B. R. de Supinski, "Exploring hardware overprovisioning in power-constrained, high performance computing," in *Proceedings of the 27th International ACM Conference on Supercomputing (ICS '13)*. ACM, 2013, pp. 173–182.

[17] J. You, J.-W. Chung, and M. Chowdhury, "Zeus: Understanding and optimizing GPU energy consumption of DNN training," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023, pp. 119–139. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/you

[18] J. Pool, A. Lastra, and M. Singh, "Energy-precision tradeoffs in mobile graphics processing units," in *IEEE International Conference on Computer Design (ICCD)*, 2008, pp. 60–67.

[19] A. Angerd, E. Sintorn, and P. Stenström, "A framework for automated and controlled floating-point accuracy reduction in graphics applications on GPUs," *ACM Transactions on Architecture and Code Optimization*, vol. 14, no. 4, 2017, article 46.

[20] R. Jahadi and E. Atoofian, "Low-power register file for tensor cores," in *IEEE 15th International Green and Sustainable Computing Conference (IGSC)*, 2024, pp. 159–164.

[21] N. Chatterjee *et al.*, "Architecting an energy-efficient DRAM system for GPUs," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 73–84.

[22] M. O'Connor *et al.*, "Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 41–54.

[23] S. Ghose *et al.*, "What your DRAM power models are not telling you: Lessons from a detailed experimental study," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 3, 2018.

[24] C. Rubio-González *et al.*, "Precimonious: Tuning assistant for floating-point precision," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC13)*, 2013, pp. 1–12.

[25] N. J. Higham and T. Mary, "Mixed precision algorithms in numerical linear algebra," *Acta Numerica*, vol. 31, pp. 347–414, 2022.

[26] T. Grützmacher, T. Cojean, G. Flegar, H. Anzt, and E. S. Quintana-Ortí, "Acceleration of PageRank with customized precision based on mantissa segmentation," *ACM Transactions on Parallel Computing*, vol. 7, no. 1, pp. 1–19, 2020.

[27] A. C. I. Malossi *et al.*, "The transprecision computing paradigm: Concept, design, and applications," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 1105–1110.

[28] A. Haidar, S. Tomov, J. Dongarra, and N. J. Higham, "Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC18)*, 2018, pp. 603–613.

[29] J. Dongarra and P. Luszczek, "HPL-MxP benchmark: Mixed-precision algorithms, iterative refinement, and scalable data generation," *The International Journal of High Performance Computing Applications*, vol. 40, no. 1, pp. 52–62, 2026, published online Sep. 25, 2025; arXiv:2509.19618.

[30] M. Klöwer, M. Razinger, J. J. Dominguez, P. D. Düben, and T. N. Palmer, "Compressing atmospheric data into its real information content," *Nature Computational Science*, vol. 1, no. 11, pp. 713–724, 2021.

[31] K. Ozaki, T. Ogita, S. Oishi, and S. M. Rump, "Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications," *Numerical Algorithms*, vol. 59, no. 1, pp. 95–118, 2012.

[32] H. Ootomo, K. Ozaki, and R. Yokota, "DGEMM on integer matrix multiplication unit," *The International Journal of High Performance Computing Applications*, vol. 38, no. 4, pp. 297–313, 2024.

[33] NVIDIA, "cuBLAS library documentation, section 1.5: Floating point emulation (v13.4; archived 13.2.1)," https://docs.nvidia.com/cuda/cublas/index.html#floating-point-emulation, accessed Oct. 5, 2026.

[34] A. Schwarz *et al.*, "Guaranteed DGEMM accuracy while using reduced precision tensor cores through extensions of the Ozaki scheme," in *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region (SCA/HPCAsia)*, 2026, pp. 91–101.

[35] S. Abdulah, H. Ltaief, Y. Sun, M. G. Genton, and D. E. Keyes, "Geostatistical modeling and prediction using mixed precision tile Cholesky factorization," in *IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2019, pp. 152–162.

[36] S. Abdulah *et al.*, "Accelerating geostatistical modeling and prediction with mixed-precision computations: A high-productivity approach with PaRSEC," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 964–976, 2022.

[37] Q. Cao *et al.*, "Reshaping geostatistical modeling and prediction for extreme-scale environmental applications," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022, pp. 1–12.

[38] Q. Cao, S. Abdulah, H. Ltaief, M. G. Genton, D. Keyes, and G. Bosilca, "Reducing data motion and energy consumption of geospatial modeling applications using automated precision conversion," in *IEEE International Conference on Cluster Computing (CLUSTER)*, 2023, pp. 330–342.

[39] J. Ren, H. Ltaief, S. Abdulah, and D. E. Keyes, "Accelerating mixed-precision out-of-core Cholesky factorization with static task scheduling,"

in *ISC High Performance 2025 Research Paper Proceedings (40th International Conference)*, 2025, pp. 1–12.
[40] J. Woo, S. Kim, and W. Y. Kim, "Dynamic precision approach for accelerating large-scale eigenvalue solvers in electronic structure calculations on graphics processing units," *Journal of Chemical Theory and Computation*, vol. 19, no. 5, pp. 1457–1465, 2023.
[41] NVIDIA, "NVIDIA GH200 Grace Hopper superchip datasheet," Document 3773000, linked from https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/, Mar. 2025, accessed Oct. 5, 2026.
[42] HPL-MxP, "HPL-MxP rules," https://hpl-mxp.org/rules.md, accessed Oct. 5, 2026.
[43] P. Giannozzi *et al.*, "Quantum ESPRESSO toward the exascale," *The Journal of Chemical Physics*, vol. 152, p. 154105, 2020.
[44] X. Wu *et al.*, "Enhancing GPU-acceleration in the Python-based simulations of chemistry frameworks," *WIREs Computational Molecular Science*, vol. 15, no. 2, p. e70008, 2025.
[45] TOP500.org, "The Linpack benchmark," https://www.top500.org/project/linpack/, accessed Oct. 5, 2026.